The Effects of AI-Assisted Software Engineering on Code Quality and Security

Motivation

We study how AI coding assistants affect code quality and security in real projects. AI made writing code cheap, but checking it is not. A large share of AI generated code still has high severity vulnerabilities, and that has not significantly improved as the tools get better [1].

Adoption is already near universal. The Google DORA 2025 report puts it at 90% of software professionals, median two hours a day, and finds a negative correlation with software delivery stability [9]. A peer reviewed study of 517 programming questions to ChatGPT found that 52% of its answers contained incorrect information, yet users still preferred them 35% of the time and missed the errors in 39% of cases [10].

Teams that adopt AI ship more code than reviewer capacity can handle, so the bottleneck is moving from writing to reviewing [2]. This report frames the two challenges with industry cases, then tests the quality-debt claim on a like-for-like AI-versus-human implementation under SonarQube.

Challenge 1: Quality and security drift

What AI puts into the codebase is buggier and more duplicated than what it replaced [1, 3]. We found two independent cases in the industry:

- (*Failure case*) **Clawdbot / OpenClaw** Viral open source AI agent project. During a forced rename, crypto scammers seized the GitHub organisation in 10 seconds. Running instances were found exposed via Shodan with plaintext credentials, and viral growth meant nobody had time to fix it. Every prevention tool that should have caught this already existed, but none operated at the speed required [4, 5].
- (*Working case*) **GitHub Dependabot at scale.** Github Octoverse 2025 says that across 2.66 million repositories with Dependabot enabled, automated dependency vulnerability fix times collapsed from 200 to 300 days down to 30 to 50 days [6]. A working example of automation closing a security gap at scale.

Challenge 2: Review cannot keep up

These quality problems get worse because review cannot keep up. When bad code is produced, the people meant to catch it are buried in volume [2]. Open source maintainers are hit hardest, and this is how the bad code reaches production. We found two independent cases in the industry:

- (*Failure case*) **curl bug bounty closure (2025).** curl had a bug report program where people could report security bugs for a reward. The program got flooded with AI generated reports that sounded professional but were almost all wrong. The maintainer tried adding stricter rules and examples of what not to submit, but nothing helped, so he shut the program down entirely [2, 7].
- (*Working case*) **Google's internal review tooling.** Google's code review tooling reports 97% engineer satisfaction. They have layered ML on top: an AutoCommenter trained on around 3 billion examples to flag best practice violations, and a "critic" feature in their Jules coding assistant that critiques AI generated code while it is being produced [8].

Discussion

Both working cases only cover part of their challenge. Dependabot fixes known dependency vulnerabilities, but the Clawdbot case was exposed credentials and open interfaces, things no dependency scanner would catch. Google's review tooling shows what automation can do, but it depends on Google's scale, something a one person project like curl cannot replicate. No single solution covers a whole challenge on its own, and the ones that exist require resources the projects most at risk do not have.

Question. Does the quality and security debt the literature reports show up on a like-for-like AI-versus-human implementation of the same specification under one static analyser? The case study below tests this.

Case study

To probe the gaps above, we take the same specification and produce two implementations, then apply one static analysis tool to both and compare findings. This directly tests Challenge 1 (quality drift); Challenge 2 (review bottleneck) is addressed indirectly through the production-versus-verification asymmetry the comparison reveals.

Specification. PM1 Project 3, a first-semester ZHAW Java assignment: a text version of Settlers of Catan. Scope includes the base game, a cities extension via inheritance, and a random victim robber variant. 30 assignment points in total.

Two implementations. The human side is a frozen snapshot of a public first-semester student team's implementation (boostvolt/zhaw-catan), used with the author's permission. The AI side is generated by Claude Opus from a consolidated English version of the original spec, with the same scope, the same fixed helper library (`ch.zhaw.hexboard`), and the same first-semester Java subset. Both sides build as a Maven project and carry the three required `requirement*` unit tests named in the original assignment.

Tool selection. Two phases. First, each candidate from GitHub's security stack was run against a Node.js bench project built to trigger it: an outdated CVE bearing dependency for Dependabot and Dependency Review, a hardcoded API key for Secret Protection, a coding bug for CodeQL, and an open PR for Copilot code review. The bench confirmed each tool works in its native habitat and that the Catan study sits outside it: two static Java snapshots, no PR, no secrets, almost no dependencies.

Tool	Surface	Fit for Catan	Verdict
Dependabot	Manifests, lockfiles	Tiny dependency surface — nothing to flag	Discarded
GitHub Advanced Security / CodeQL	Source code	Strong on Java but security only; misses maintainability and reliability signal	Runner-up
Secret Protection	Commits, history	Offline game, no secrets	Discarded
Copilot code review	PR diff	No active PR	Discarded
Dependency Review	PR manifest diff	No PR, no manifest change	Discarded
SonarQube	Full Java Maven project	Bugs, reliability, maintainability, hotspots in one scan	Selected

SonarQube (community edition, run locally against the Maven build) was selected for the Catan run. It is a static code analyser with first class Java and Maven support that exposes bugs, reliability, maintainability, and security hotspots in a single scan without needing a PR diff or a dependency change event. CodeQL was the runner up; SonarQube won on broader signal because this study is about quality debt, not just vulnerabilities. The Node.js bench justifies the tool choice; Catan tests the literature claim that AI generated code carries more quality and security debt than human written code of equivalent scope.

Structural comparison. The AI version is materially smaller than the human version at the same specification. Main source is 1,108 lines across 9 files against the human's 2,718 across 17; test source is 260 lines in a single test file against 2,060 across twelve (13%). The AI omits a Road class (roads are two-character strings on hex edges), omits an abstract Structure base, and concentrates most logic in a single `SiedlerGame` class of 435 lines (39% of main source). Claude Opus produced this runnable 30-point implementation in roughly ten minutes of wall-clock time; the human repository's commit history spans weeks. The asymmetry in production speed relative to verification surface (test-to-main LOC ratio of 0.23 against 0.76) is the motivating gap shown in miniature: AI-assisted production is much cheaper than AI-assisted verification.

Scan findings. SonarQube was run against both codebases on 2026-05-01. Both projects passed the configured quality gate and reported zero direct security issues. The signal is on reliability and maintainability.

Metric	AI	Human
Lines of code	1.6k	2.0k
Security rating / issues	A / 0	A / 0
Reliability rating / issues	D / 3	C / 1
Maintainability rating / issues	A / 16	A / 9
Security hotspots	3	1
Coverage	0.0%	0.0%
Duplications	0.0%	0.0%

Coverage reads 0% on both because neither Maven build emits a JaCoCo report; the comparison here is on static signal only. Normalised by SonarQube's own LOC figure, the AI version carries about 10.0 maintainability issues per kLOC against the human's 4.5. On reliability the gap is wider: 1.9 issues per kLOC against 0.5.

The four security hotspots across both projects are all the same rule (`java:S2245`, weak PRNG); for an offline Catan game these are almost certainly benign uses of `Random`, not cryptographic risk.

The AI's reliability findings concentrate in `SiedlerGame`: two of three are the same "save and re-use this `Random`" anti-pattern, and the third sits in the shared `ch.zhaw.hexboard` library — which is also reported on the human side, so net of that shared library the AI carries 2 unique reliability issues to the human's 0.

Its maintainability list is dominated by "Brain Method" cognitive complexity findings on `App.java` (33) and `SiedlerGame` (up to 49, against an allowed 15) plus repeated `Map`-to-`EnumMap` suggestions. The human side has no cognitive complexity findings; its maintainability list is mostly style, deprecation hygiene, and small API conformance items.

Interpretation. The scan partially supports the literature claim: quality drift is visible, but the security delta is effectively zero.

Supported on reliability and maintainability. The AI implementation scores worse on both axes even on the per-kLOC denominator that favours the smaller codebase. The findings concentrate exactly where the structural comparison predicted — in oversized methods inside `SiedlerGame`, the same flat-package god-class shape the "code clones / missing abstraction" thread in the literature would lead us to expect.

Not supported on direct security. Both projects are effectively blank from a vulnerability perspective. That is more a property of the input than of the AI. Small offline Java student projects with tiny dependency surfaces and no real secret handling path do not expose much for vulnerability oriented tooling to bite on; CodeQL would likely report similarly little.

Caveats. Scan surface is not the same on both sides: the AI side has a test-to-main LOC ratio of 0.23 against 0.76, omits the `Road` class and the abstract `Structure` base, and ships one test file against twelve. A raw finding count is not a like-for-like comparison, which is why the density read above matters more than the absolute totals. Separately, SonarQube does not validate game

rule correctness — the five self-flagged rule-interpretation uncertainties from the AI side sit outside what static analysis can see.

Summary. On this assignment, SonarQube sees more static bug and smell pressure in the AI version, but no meaningful security delta. That maps onto Challenge 1 (quality drift) — even when an AI assistant produces a runnable, spec-passing artefact in 10 minutes against weeks of human work, the static analysis fingerprint shows the cost: concentrated complexity in core logic, less abstraction depth, thinner self-validating test surface. Challenge 2 (review bottleneck) is not directly testable in a single-artefact study, but the production-versus-verification asymmetry — 10 minutes to write 1,108 lines of main source against a test-to-main ratio of 0.23 — is the same asymmetry the curl and OpenClaw cases describe at scale.

Lessons.

- Production cost (10 minutes for 1,108 main lines) decoupled from verification cost (test-to-main 0.23). The cheap thing got cheaper; the expensive thing did not.
- Static analysis catches quality drift in core logic — cognitive complexity 49 in `SiedlerGame`, repeated god-class smells — but cannot see rule-correctness or cross-cutting failures of the curl/OpenClaw shape.
- A clean security rating on a small offline project says more about scan surface than AI behaviour. Vulnerability tooling needs an environment that triggers it: secrets, dependencies, network surface.
- Prevention tooling existed for every failure case in the literature; none of it operated at the speed or scale the projects most at risk actually had.

Limitations. Single specification, single human team, single AI model, single prompt, single tool. The findings are suggestive, not generalisable. A fuller study would require multiple prompts, multiple models, and multiple specifications.

Literature

[1] Veracode, *2025 GenAI Code Security Report* (2025). 100+ LLMs evaluated; 45% of AI generated code contains at least one high severity vulnerability, and AI generated code carries 2.74x more vulnerabilities than human written code. https://www.veracode.com/resources/analyst-reports/2025-genai-code-security-report/

[2] LeadDev, *Open Source Has a Big AI Slop Problem* (2025). 12x reviewer asymmetry. https://leaddev.com/software-quality/open-source-has-a-big-ai-slop-problem

[3] GitClear, *AI Copilot Code Quality: 2025 Data Suggests 4x Growth in Code Clones* (2025). Code duplication trends across 211M changed lines. https://www.gitclear.com/ai_assistant_code_quality_2025_resear

[4] Palo Alto Networks Unit 42, *OpenClaw May Signal the Next AI Security Crisis* (2026). Clawdbot security analysis. https://www.paloaltonetworks.com/blog/network-security/why-moltbot-may-signal-ai-crisis/

[5] Sivaram, P.G., *From Clawdbot to Moltbot* (2026). DEV Community. Clawdbot incident timeline. https://dev.to/sivarampg/from-clawdbot-to-moltbot-how-a-cd-crypto-scammers-and-10-seconds-of-chaos-took-down-the-4eck

[6] GitHub, *Octoverse 2025* (2025). Dependabot adoption and fix time data. https://github.blog/news-insights/octoverse/octoverse-a-new-developer-joins-github-every-second-as-ai-leads-typescript-to-1/

[7] The New Stack, *96% of Codebases Rely on Open Source, and AI Slop Is Putting Them at Risk* (2025). curl bug bounty context. https://thenewstack.io/ai-slop-open-source/

[8] Engineer's Codex, *How Google Takes the Pain Out of Code Reviews, with 97% Dev Satisfaction* (2025). Google Critique, AutoCommenter, Jules critic. https://read.engineerscodex.com/p/how-google-takes-the-pain-out-of

[9] Google Cloud DORA, *2025 DORA State of AI-Assisted Software Development* (2025). AI adoption at 90% of software professionals, negative correlation with software delivery stability. https://dora.dev/research/2025/dora-report/

[10] Kabir, S., Udo-Imeh, D.N., Kou, B., Zhang, T., *Is Stack Overflow Obsolete? An Empirical Study of the Characteristics of ChatGPT Answers to Stack Overflow Questions* (CHI 2024). 52% of ChatGPT answers to programming questions contain incorrect information; users prefer them 35% of the time and miss the errors 39% of the time. https://arxiv.org/pdf/2308.02312